Harvard John A. Paulson
School of Engineering
and Applied Sciences

Do Input Gradients Highlight Discriminative Features?

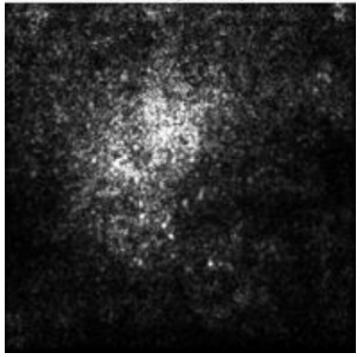
Authors: Harshay Shah, Prateek Jain, Praneeth Netrapalli
Presenters: Karly Hou, Eshika Saxena, Kat Zhang

Introduction

- Instance-specific explanations of model predictions
- Input coordinates are ranked in decreasing order of input gradient magnitude
- **Assumption (A)**: Larger input gradient magnitude = higher contribution to prediction

Image

Sensitivity map M_c

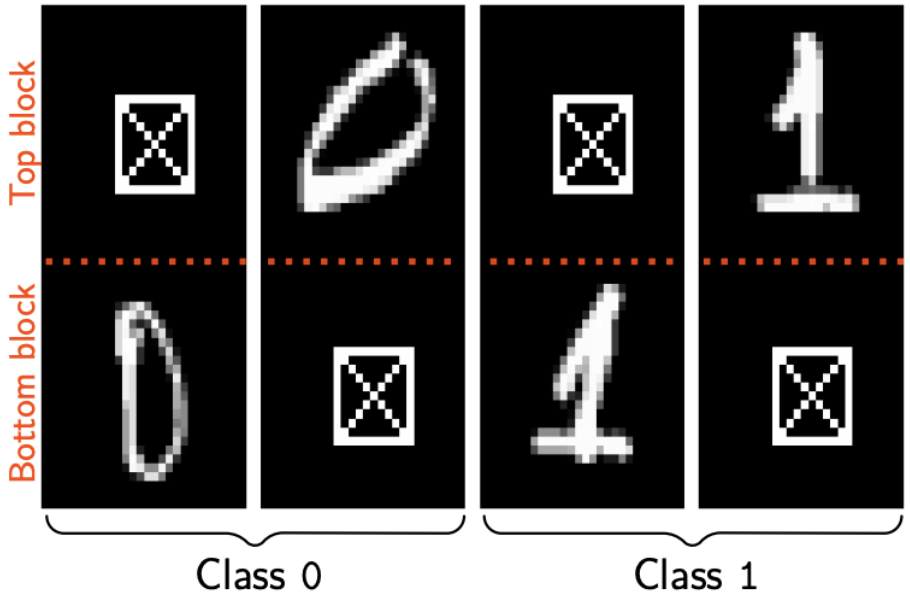


Key Contributions

- New evaluation framework **DiffROAR** for evaluating **Assumption (A)** on real datasets
  - compares the top-ranked and bottom-ranked features
- Evaluate input gradient attributions for MLPs and CNNs
- Found that standard models violate Assumption (A)
- Some adversarially trained models might satisfy it

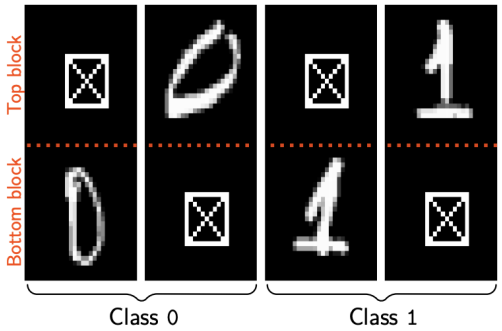
Key Contributions: BlockMNIST Dataset

- Each datapoint contains two images, a signal (the actual digit), and a null block (a blank square)
- Want to see if the classification is actually occurring as a result of the block with the information

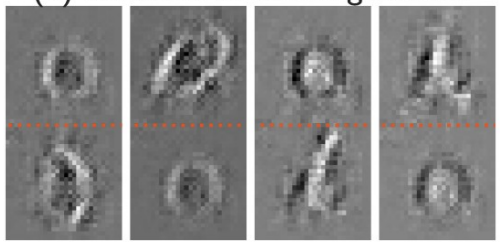


Key Contributions: BlockMNIST Dataset

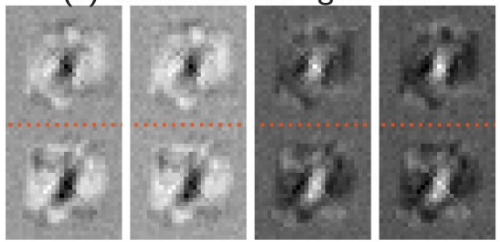
- Feature leakage is a potential reason why Assumption (A) is violated
- Feature leakage is shown empirically and proved theoretically



(b) Standard Resnet18 gradients



(c) Standard MLP gradients



Related Work

- Sanity checks for explanations
  - Visual assessments are unreliable
  - Other well-known gradient-based attribution methods fare worse in these sanity checks

- Evaluating explanation fidelity
  - Evaluations traditionally performed without knowing the ground-truth explanations
  - Usually evaluated using masking or ROAR
  - Input gradients lack explanatory power and are as good as random attributions

Related Work

- Adversarial Robustness
  - Adversarial training can improve the visual quality of input gradients
  - Adversarial model gradients are more stable than standard ones
  - Are they also more faithful?

DiffROAR Evaluation Framework

Setting

- Standard classification setting with each independently drawn data point as a pair of (instance, label) $(x^{(i)}, y^{(i)})$
- $x^{(i)}_j$ denotes jth coordinate/feature of $x^{(i)}$
- **Feature attribution scheme** $\mathcal{A} : \mathbb{R}^d \to \{\sigma : \sigma \text{ is a permutation of } [d]\}$

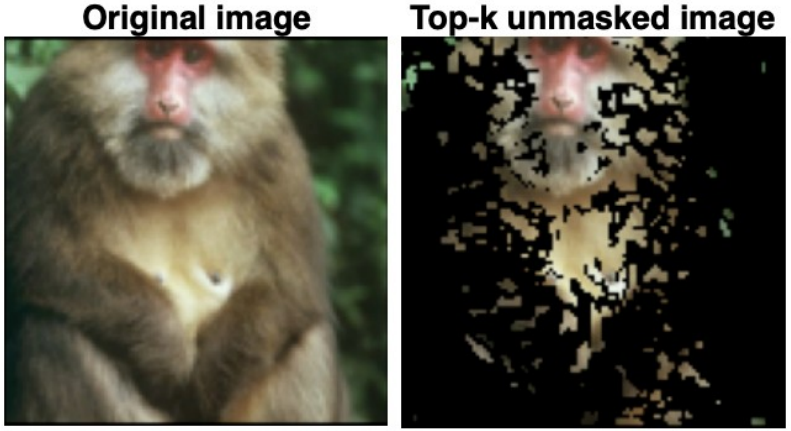
  maps a d-dimensional instance x to a permutation of its features
- e.g. **Input gradient attribution scheme** takes input instance x, predicted label y, outputs ordering [d] that ranks features in decreasing order of input gradient magnitude $\mathcal{A}(x) : [d] \to [d]$

Unmasking Schemes

- **Unmasked instance** x^S zeroes out all coordinates not in subset S
- **Unmasking scheme** maps instance x to subset A(x) of coordinates to obtain unmasked instance $x^{A(x)}$

$$A : \mathbb{R}^d \to \{S : S \subseteq [d]\}$$

- To $\mathcal{A}_k^{\text{top}}(x) := \{\mathcal{A}(x)_j : j \leq k\}$, ng scheme
$\mathcal{A}_k^{\text{bot}}(x) := \{\mathcal{A}(x)_j : d - k < j \leq d\}$.



Predictive Power of Unmasking Schemes

- **Predictive power**: best classification accuracy that can be attained by training a model with architecture **M** on unmasked instances that are obtained via unmasking scheme **A**

$$\texttt{PredPower}_M(A) := \sup_{f \in M, f:\mathbb{R}^d \to \mathcal{Y}} \mathbb{E}_{\mathcal{D}} \left[\mathbb{1} \left[f(x^{A(x)}) = y \right] \right].$$

- Estimate predictive power in two steps:
  (1) Use unmasking scheme A to obtain unmasked train and test datasets that comprise data points of the form ($x^{A(x)}$, y)
  (2) Retrain a new model with the same architecture M on unmasked train data and evaluate its accuracy on unmasked test data

DiffROAR Metric

$$\texttt{DiffROAR}_M(\mathcal{A}, k) = \texttt{PredPower}_M(\mathcal{A}_k^{\text{top}}) - \texttt{PredPower}_M(\mathcal{A}_k^{\text{bot}})$$

Interpretation of the metric:

- Sign of the metric indicates whether the assumption is satisfied (> 0) or violated (< 0)
- Magnitude of the metric quantifies extent of separation of most and least discriminative coordinates into two disjoint subsets

Experiment 1: Image Classification Benchmarks

Setup: Datasets and Models

- **Datasets**: SVHN, Fashion MNIST, CIFAR-10, ImageNet-10
- **Models**: Standard and adversarially trained two-hidden-layer MLPs and Resnets
  - Adversarial: l_2 and l_∞ epsilon-robust models with perturbation budget epsilon using PGD adversarial training

Procedure: Computing DiffROAR Metric

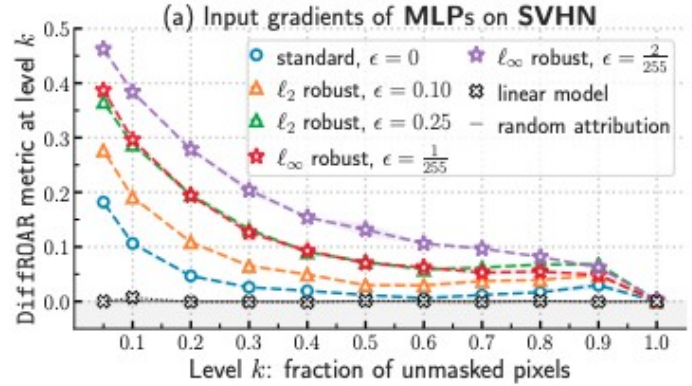
As a function of unmasking fraction k, compare input gradient attributions of models to baselines of model-agnostic and input-agnostic attributions:

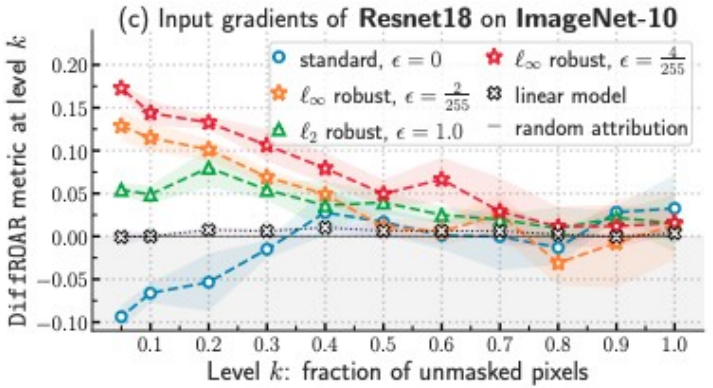
1. Train a standard or robust model with architecture M on the original dataset and obtain its input gradient attribution scheme A.
2. Use attribution scheme A and level k (i.e., fraction of pixels to be unmasked) to extract the top-k and bottom-k unmasking schemes: A top k and A bot k

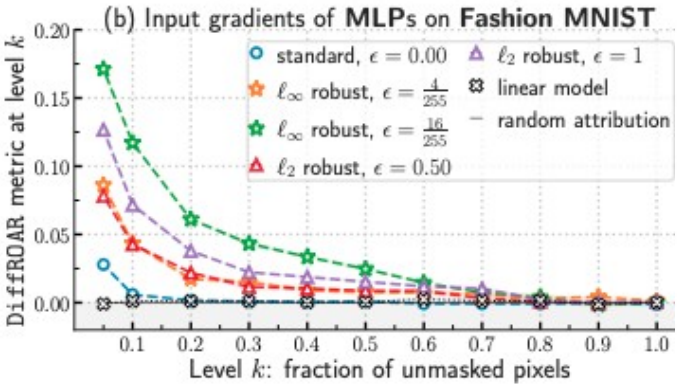
Procedure: Computing DiffROAR Metric

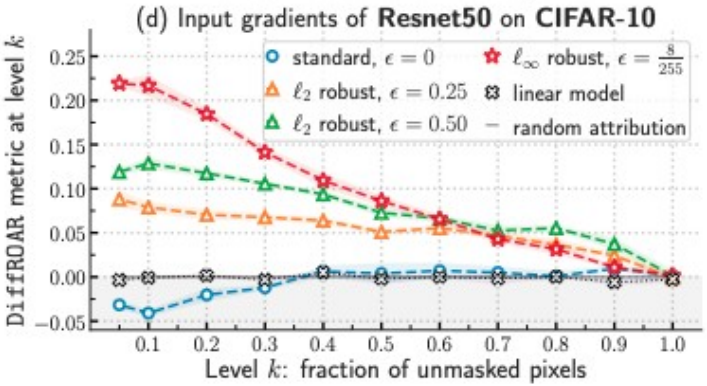
3. Apply A top k and A bot k on the original train & test datasets to obtain top-k and bottom-k unmasked datasets respectively (unmask individual image pixels without grouping them channel-wise)
4. Estimate top-k and bottom-k predictive power by retraining new models with architecture M on top-k and bottom-k unmasked datasets respectively and compute the DiffROAR metric.
5. Average the DiffROAR metric over five runs for each model and unmasking fraction or level k

Results

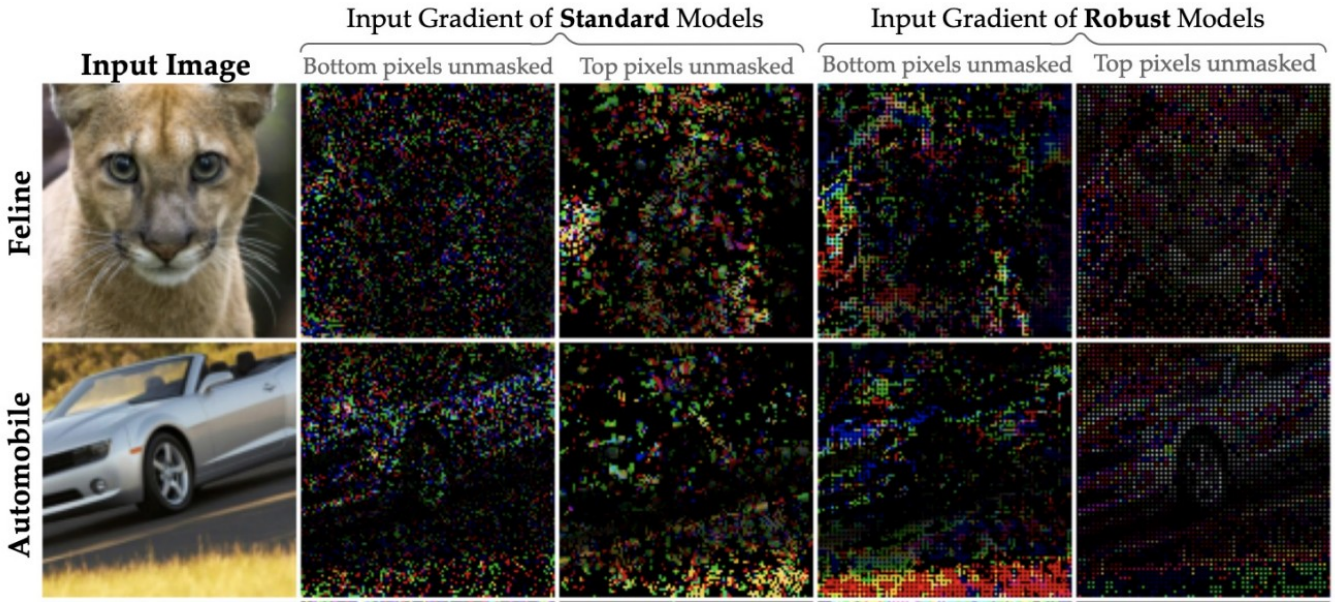

(a) Input gradients of MLPs on SVHN
standard, $\epsilon = 0$
ℓ_2 robust, $\epsilon = 0.10$
ℓ_2 robust, $\epsilon = 0.25$
ℓ_∞ robust, $\epsilon = \frac{1}{255}$
ℓ_∞ robust, $\epsilon = \frac{2}{255}$
linear model
random attribution
DiffROAR metric at level k
Level k: fraction of unmasked pixels


(c) Input gradients of Resnet18 on ImageNet-10
standard, $\epsilon = 0$
ℓ_∞ robust, $\epsilon = \frac{2}{255}$
ℓ_2 robust, $\epsilon = 1.0$
ℓ_∞ robust, $\epsilon = \frac{4}{255}$
linear model
random attribution
DiffROAR metric at level k
Level k: fraction of unmasked pixels


(b) Input gradients of MLPs on Fashion MNIST
standard, $\epsilon = 0.00$
ℓ_∞ robust, $\epsilon = \frac{4}{255}$
ℓ_∞ robust, $\epsilon = \frac{16}{255}$
ℓ_2 robust, $\epsilon = 0.50$
ℓ_2 robust, $\epsilon = 1$
linear model
random attribution
DiffROAR metric at level k
Level k: fraction of unmasked pixels


(d) Input gradients of Resnet50 on CIFAR-10
standard, $\epsilon = 0$
ℓ_2 robust, $\epsilon = 0.25$
ℓ_2 robust, $\epsilon = 0.50$
ℓ_∞ robust, $\epsilon = \frac{8}{255}$
linear model
random attribution
DiffROAR metric at level k
Level k: fraction of unmasked pixels

Results - Qualitative Analysis



Experiment 2: BlockMNIST Data

Analyzing input gradient attributions: BlockMNIST data

Dataset design based on intuitive properties of classification tasks:

- Object of interest may appear in **different parts** of an image
- Object of interest and the rest of the image often share low-level patterns like edges that are **not informative** of the label on their own

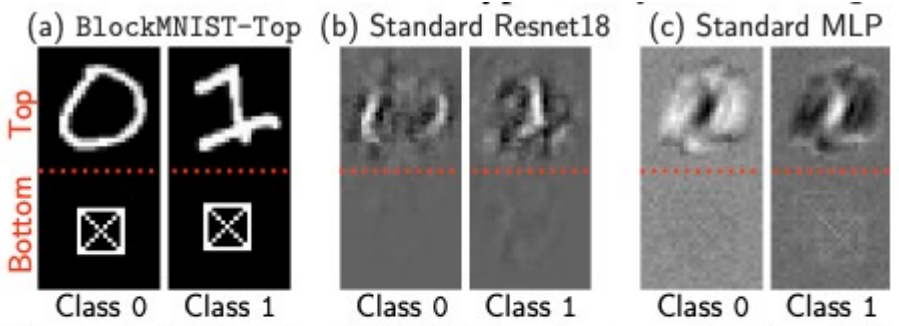
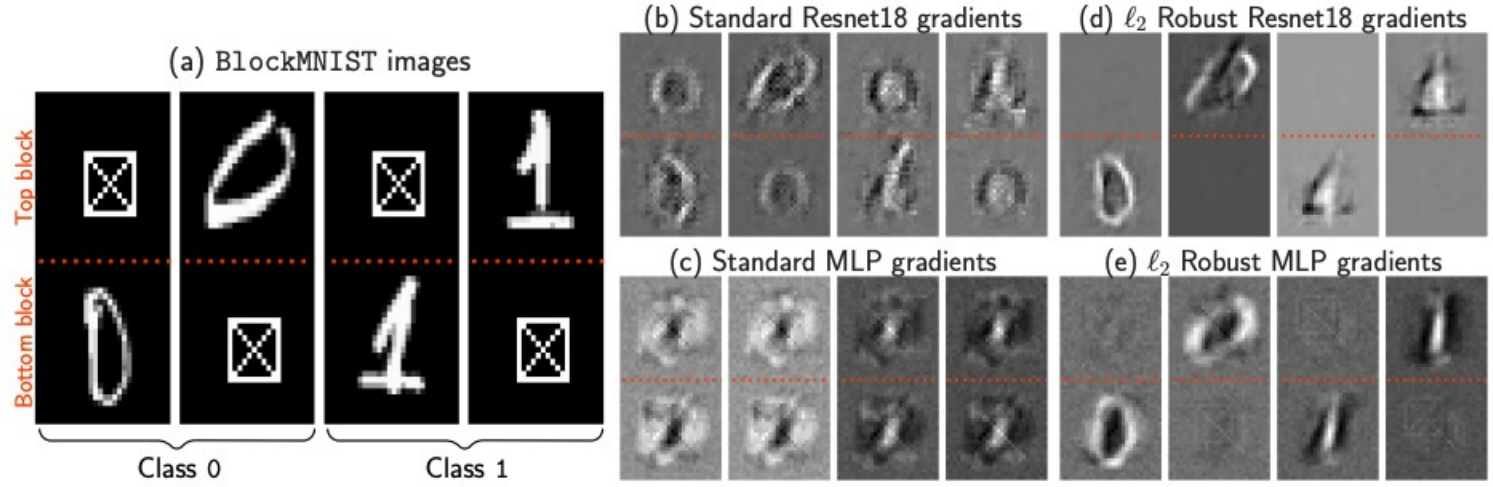


Figure 4: (a) In `BlockMNIST-Top` images, the signal & null blocks are fixed at the top & bottom respectively. In contrast to results on `BlockMNIST` in fig. 1, input gradients of standard (b) Resnet18 and (c) MLP trained on `BlockMNIST-Top` highlight discriminative features in the signal block, suppress the null block, and satisfy (A).

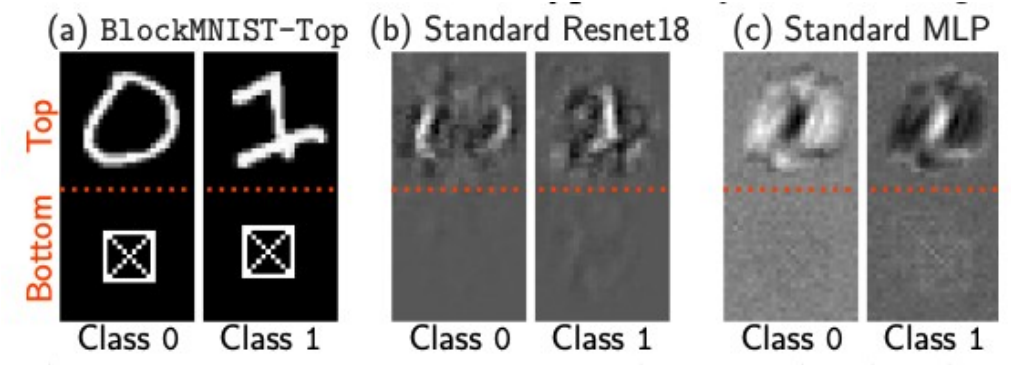
Standard + adversarially robust models trained on BlockMNIST data attain 99.99% test accuracy.

Do gradient attributions highlight signal block over null block? Not always!



Feature leakage hypothesis

- When discriminative features vary across instances (e.g. signal block at top vs bottom), input gradients of standard models may not only highlight instance-specific features but also **leak discriminative features** from other instances
- Altered datasets: BlockMNIST-Top → now gradients of standard Resnet18, MLP highlight discriminative features in signal block and suppress null block

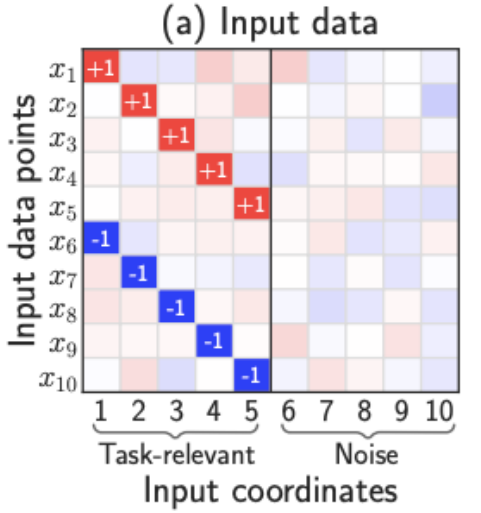


Proof 3: Feature Leakage

Dataset

- Simplified version of BlockMNIST
- Draw sample $(x, y) \in \mathbb{R}^{d \cdot d} \times \{\pm 1\}$ $y = \pm 1$ with probability 0.5 and $x = [\eta g_1, \quad \eta g_2, \quad \ldots, \quad yu^* + \eta g_j, \quad \ldots, \quad \eta g_d]$ with j chosen at random from $[d/2]$
  - eta = noise parameter
  - g_i drawn uniformly at random from the unit ball
  - Let d even so that d/2 is an integer
- We can think of each x as a concatenation of blocks {x_1...x_d}
  - The first d/2 blocks are task-relevant: each example contains an instance-specific signal block that is informative of its label
  - The remaining blocks are noise blocks that don't contain task-relevant signal
- At a high level, these correspond to the discriminative MNIST digit and null square patch in BlockMNIST

:



Model

- Consider one-hidden layer MLPs with ReLU nonlinearity
- Given an input instance, the output logit f and cross-entropy loss L are (for given layer width m)

$$f((w,R,b),x) := \langle w, \phi(Rx+b) \rangle, \quad \mathcal{L}((w,R,b),(x,y)) := \log\left(1+\exp\left(-y \cdot f((w,R,b),x)\right)\right)$$

- As m → infinity, the training procedure equivalent to gradient descent on infinite-dimensional Wasserstein space
- Wasserstein space: network interpreted as probability distribution nu with output score, CE loss:

$$f(\nu,x) := \mathbb{E}_{(w,r,b)\sim\nu}\left[w\phi\left(\langle r,x\rangle+b\right)\right], \quad \mathcal{L}(\nu,(x,y)) := \log\left(1+\exp\left(-y \cdot f(\nu,x)\right)\right)$$

Theoretical analysis

- Prior research shows that if gradient descent in the Wasserstein space $\mathcal{W}^2\left(\mathbb{R} \times \mathbb{R}^{\tilde{d}d} \times \mathbb{R}\right)$ on $\mathbb{E}_{\mathcal{D}}\left[\mathcal{L}(\nu, (x, y))\right]$ converges, it does to a max-margin classifier: $$\nu^* := \underset{\nu \in \mathcal{P}\left(\mathbb{S}^{d\tilde{d}+1}\right)}{\arg\max} \min_{(x,y)\sim\mathcal{D}} y \cdot f(\nu, x),$$
- S = surface of Euclidean unit ball
- P(S) = space of probability distributions
- Intuitively, our results show that on any data point (x, y) ~ D, the input gradient magnitude of the max-margin classifier v^* is equal over all task-relevant blocks and zero on noise blocks

Theorem

Theorem 1. *Consider distribution $\mathcal{D}$ (2) with $\eta < \frac{1}{10d}$. There exists a max-margin classifier ν^* for $\mathcal{D}$ in Wasserstein space (i.e., training both layers of FCN with $m \to \infty$) given by (4), such that for all $\forall\ (x, y) \sim \mathcal{D}$: (i) $\left\|\left(\nabla_x \mathcal{L}(\nu^*, (x, y))\right)_j\right\| = c > 0$ for every $j \in [d/2]$ and (ii) $\left\|\left(\nabla_x \mathcal{L}(\nu^*, (x, y))\right)_j\right\| = 0$ for every $j \in \{d/2 + 1, \cdots, d\}$, where $\left(\nabla_x \mathcal{L}(\nu^*, (x, y))\right)_j$ denotes the j^{th} block of the input gradient $\nabla_x \mathcal{L}(\nu^*, (x, y))$.*

- Guarantees **existence of max-margin classifier** such that the input gradient magnitude for any given instance is a nonzero constant on the task-relevant blocks, and zero on noise blocks
- However, input gradients fail at highlighting the **unique instance-specific signal block** over the task-relevant blocks
- **Feature leakage:** input gradients highlight task-relevant features that are not specific to the given instance

Empirical results

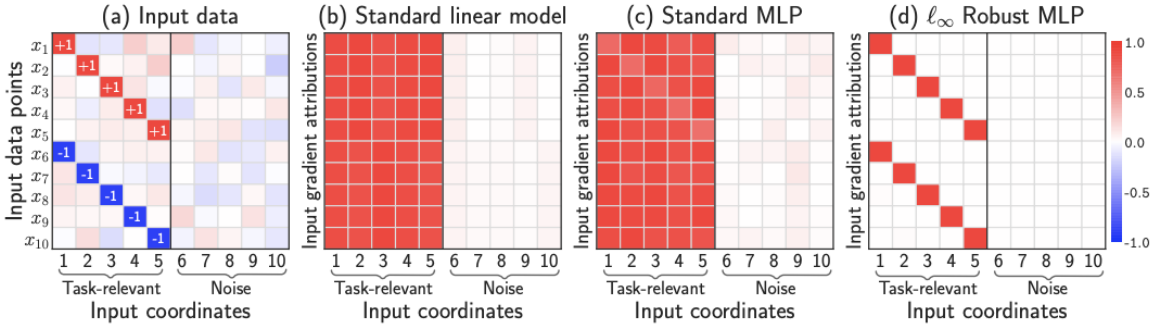


Figure 5: Input gradients of linear models and standard & robust MLPs trained on data from eq. (2) with $d = 10, \widetilde{d} = 1, \eta = 0$ and $u^* = 1$. (a) Each row in corresponds to an instance x, and the highlighted coordinate denotes the signal block $j^*(x)$ & label y. (b) Linear models suppress noise coordinates but lack the expressive power to highlight instance-specific signal $j^*(x)$, as their input gradients in subplot (b) are identical across all examples. (c) Despite the expressive power to highlight instance-specific signal coordinate $j^*(x)$, input gradients of standard MLPs exhibit feature leakage (see Theorem 1) and violate (A) as well. (d) In stark contrast, input gradients of adversarially trained MLPs suppress feature leakage and starkly highlight instance-specific signal coordinates $j^*(x)$.

- One-hidden-layer ReLU MLPs with width 10000
- All models obtain **100% test accuracy**
- Due to insufficient expressive power, linear models have input-agnostic gradients that suppress noise but **do not differentiate instance-specific signal coordinates**

Conclusion & Discussion

Discussion and Limitations

- Assuming that the model trained on the unmasked dataset learns the same features as the model trained on the original dataset
- When all features are equally informative, ROAR/DiffROAR can’t be used
- This work only focuses on “vanilla” input gradients
- Why does adversarial training actually mitigate feature leakage?

Conclusion

Assumption (A): Larger input gradient magnitude = higher contribution to prediction

- **Assumption (A)** is not necessarily true for standard models
- Adversarially robust models satisfy **Assumption (A)** consistently
- Feature leakage is the reason why **Assumption (A)** does not hold

Discussion Questions

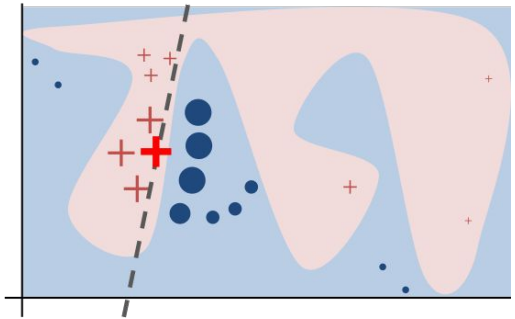
- After seeing that the fundamental assumption regarding the correspondence between input gradients and feature importance may not hold, **do we still find post-hoc explanations for individual or groups of data points convincing?**
- What are other major assumptions in explainability that we've discussed that you think could benefit from a sanity check with an **approach similar to this paper?**
- Do you have any additional **doubts** about any of the approaches taken in this paper?

On the Privacy Risks of Algorithmic Recourse

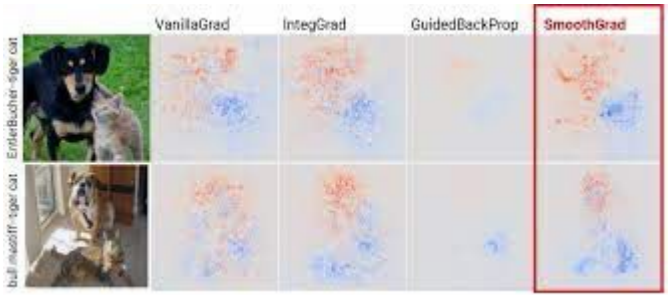
Pawelczyk, Lakkaraju, Neel

Presented by Christina Xiao, Lucas Monteiro Pas, Catherine Huang

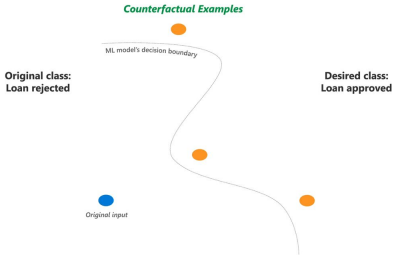
MOTIVATION



Lime

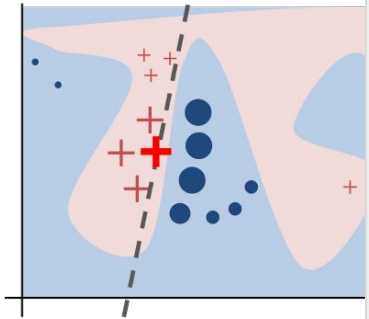


Gradient Based



Counterfactuals

MOTIVATION



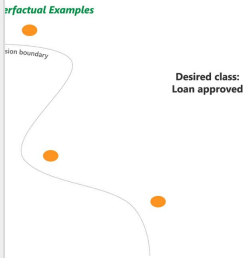
Lime

Can these methods leak:

1 - User's sensitive information?

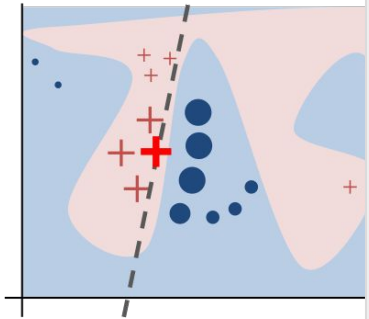
2 - Model's weights?

3 - Training dataset?



Counterfactuals

MOTIVATION



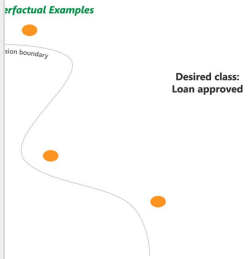
Lime

Can these methods leak:

1 - User's sensitive information?

2 - Model's weights?

3 - Training dataset?

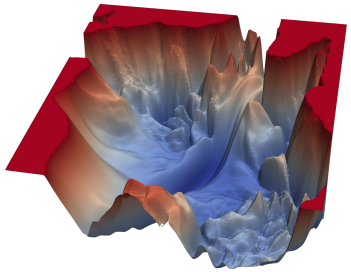


Counterfactuals

Previous works

Membership Inference.

Given access to an instance



Loss information

Instance is in training

PREVIOUS WORKS

Membership Inferen

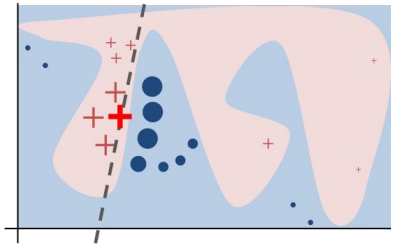
Given access to an ins nce is in training

How to leverage XAI?

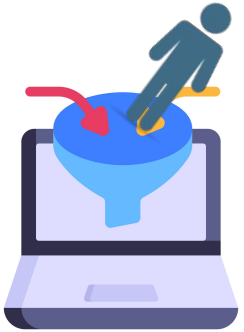
PREVIOUS WORKS

How can feature attribution impact membership inference? *Shokri, 2021.*

Given access to an instance



Feature attribution



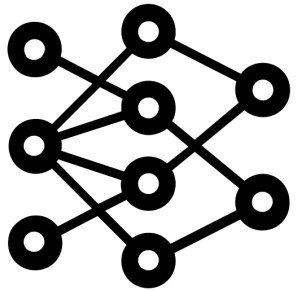
Instance is in training

Reza Shokri, Martin Strobel, and Yair Zick. On the privacy risks of model explanations. In Proceedings of the 2021 AAAI/ACM Conference on AI, Ethics, and Society (AIES), page 231–241, 2021.

PREVIOUS WORKS

Model Extraction.

Given access to predictions

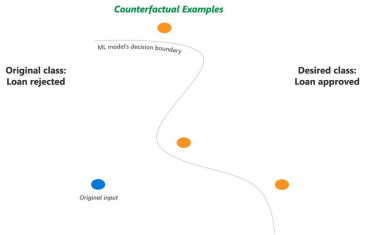


Reconstruct the model

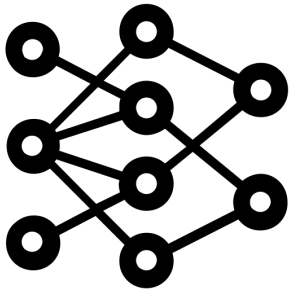
PREVIOUS WORKS

How can counterfactual explanations impact model extraction? *Aïvodji, 2020.*

Given access to predictions



Counterfactual explanations



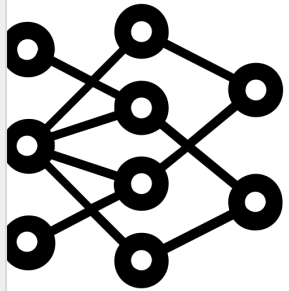
Reconstruct the model

Ulrich Aïvodji, Alexandre Bolot, and Sébastien Gambs. Model extraction from counterfactual explanations. arXiv preprint arXiv:2009.01884, 2020.

PREVIOUS WORKS

How can counterfactual explanations impact model extraction? *Aïvodji, 202*

The authors assume that the adversary can query the models multiple times!

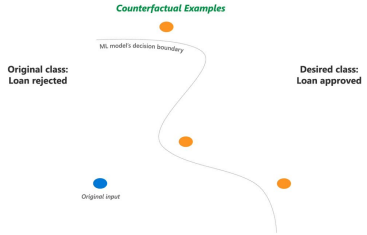


Given access to predic construct the model

Ulrich Aïvodji, Alexandre Bolot, and Sébastien Gambs. Model extraction from counterfactual explanations. arXiv preprint arXiv:2009.01884, 2020.

CONTRIBUTIONS

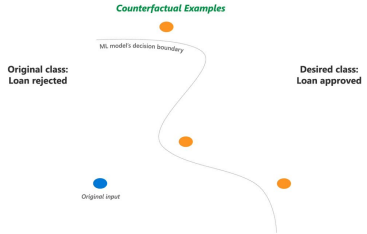
Given access to an instance



Counterfactual explanations

CONTRIBUTIONS

Membership Inference.



Given access to an instance

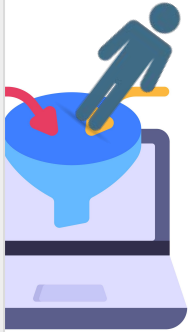
Counterfactual explanations

Instance is in training

Contributions

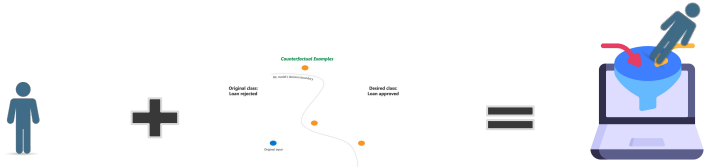
Given access to an ins

The adversary can query the models a unique time!



nce is in training

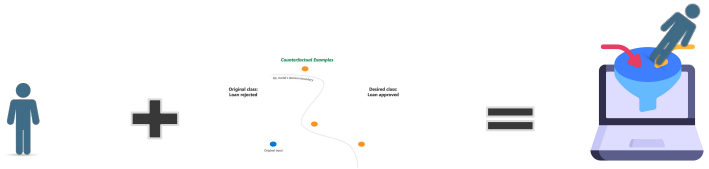
CONTRIBUTIONS



1. Define a **new class of attacks** called counterfactual distance-based attacks

2. Provide two examples of attacks in the class

Contributions



1. Define a **new class of attacks** called counterfactual distance-based attacks

2. Provide two examples of attacks in the class

c (“Instance”, “Counterfactual”)

=

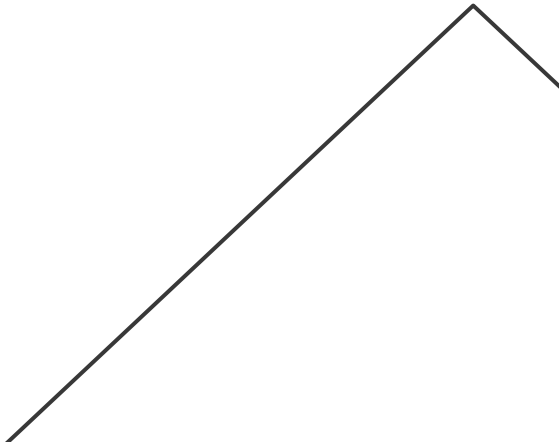
Instance is in training

Preliminaries: Algorithmic Recourse

$$x' = \underset{x' \in \mathcal{A}^p}{\arg\min}\, \ell(f_{\boldsymbol{\theta}}(x'), 1) + \lambda \cdot c(x, x')$$

Wachter et al.

PRELIMINARIES: MI ATTACKS



PRELIMINARIES: MI ATTACKS

THRESHOLDING ON LOSS (YEOM ET AL.)

loss of model with params θ on instance $z = (x, y)$

threshold

$$M_{\text{Loss}}(x) = \begin{cases} \textit{MEMBER} & \text{if } \ell(\boldsymbol{\theta}, z) \leq \tau_L \\ \textit{NON-MEMBER} & \text{if } \ell(\boldsymbol{\theta}, z) > \tau_L. \end{cases}$$

very powerful and simple, but only feasible when can access instance's y; model's ℓ, θ; underlying data distribution $\mathscr{D}$ (to practically get τ)

Preliminaries: MI Attacks

Thresholding on Loss (Yeom et al.)

loss of model with params θ on instance $z = (x, y)$

threshold

$$M_{\text{Loss}}(x) = \begin{cases} \textit{MEMBER} & \text{if } \ell(\boldsymbol{\theta}, z) \leq \tau_L \\ \textit{NON-MEMBER} & \text{if } \ell(\boldsymbol{\theta}, z) > \tau_L. \end{cases}$$

very powerful and simple, but only feasible when can access instance's y; model's ℓ, θ; underlying data distribution $\mathscr{D}$ (to practically get τ)

Loss Likelihood Ratio Attack (Carlini et al.)

Given: sample access to underlying data distribution $\mathscr{D}$

1. Adversary trains shadow models
2. Computes confidence in each model f_θ when z **in/out** train set
3. Fits normal distributions to these in/out confidences
4. Computes approximate likelihood ratio Λ
5. Predicts MEMBER when $\Lambda > \tau$

Setting: Recourse-based MI Game

owner $\mathcal{O}$ and adversary $\mathcal{A}$

Setting: Recourse-based MI Game

owner $\mathcal{O}$:

1. Draws training set D_t from underlying data distribution $\mathcal{D}$
2. Trains model f_θ
3. Labels every point z in D_t with binary label $f_\theta(z)$
4. Flips coin to determine where to sample x from
   a. If heads, conditional distribution $\mathcal{D} \mid f_\theta(z) = 0$
   b. If tails, subset of D_t with label 0
5. Generates recourse x' for x
6. Sends (x', x)

all data labelled 0 (unfavorable outcome), but combination of training data or not

Setting: Recourse-based MI Game

owner $\mathcal{O}$:

1. Draws training set D_t from underlying data distribution $\mathcal{D}$
2. Trains model f_θ
3. Labels every point z in D_t with binary label $f_\theta(z)$
4. Flips coin to determine where to sample x from
   a. If heads, conditional distribution $\mathcal{D} \mid f_\theta(z) = 0$
   b. If tails, subset of D_t with label 0
5. Generates recourse x' for x
6. Sends (x', x)

all data labelled 0 (unfavorable outcome), but combination of training data or not

adversary $\mathcal{A}$:

1. Can also query $\mathcal{D}$, knows implementation details of $\mathcal{O}$
2. Guesses whether x is MEMBER (in D_t) or NON-

Intuition

Loss-based attacks are good at determining MEMBER, because model typically overfits to training points

↓

This may be because, during training, decision boundary is pushed away from training points (Shroki et al.)

↓

Points in training set should be further from boundary than points in test set

↓

Counterfactual distance-based attacks!

ATTACK 1: THRESHOLDING ON CFD

counterfactual distance between x, x', from whichever recourse method

threshold

$$M_{\text{Distance}}(x) = \begin{cases} \texttt{MEMBER} & \text{if } c(x, x') \geq \tau_D \\ \texttt{NON-MEMBER} & \text{if } c(x, x') < \tau_D. \end{cases}$$

(recall: $M_{\text{Loss}}(x) = \begin{cases} \texttt{MEMBER} & \text{if } \ell(\boldsymbol{\theta}, z) \leq \tau_L \\ \texttt{NON-MEMBER} & \text{if } \ell(\boldsymbol{\theta}, z) > \tau_L. \end{cases}$)

assume that $\mathscr{A}$ knows a priori optimal threshold τ_α that maximizes TPR given FPR α; in practice, will plot TPR v. FPR over all τ_D

ATTACK 2: CFD LRT

again, similar to preliminaries, but more adjustments

Algorithm 1 One-sided Distance-based Likelihood Ratio Test (`CFD LRT`)

1: **Inputs:** point (x, y), recourse output s = `GetRecourse`$(x, f_{\theta}), \mathcal{D}$; FP-Rate: α, # Shadow Models: N, $\mathcal{T}$ = `TrainClassifier`$(\cdot)$
2: teststats = $[\,]$
3: **Compute**: $t_0 = T(s) = c(x, x')$ ← compute CFD on input

ATTACK 2: CFD LRT

again, similar to preliminaries, but more adjustments

Algorithm 1 One-sided Distance-based Likelihood Ratio Test (`CFD LRT`)

Inputs: point (x, y), recourse output s = `GetRecourse`$(x, f_{\boldsymbol{\theta}}), \mathcal{D}$; FP-Rate: α, # Shadow Models: N, $\mathcal{T}$ = `TrainClassifier`$(\cdot)$

teststats = $[\,]$

Compute: $t_0 = T(s) = c(x, x')$ ← compute CFD on input

for $i = 1 : N$ **do**

  Sample $\mathcal{D}_t^{(i)} \sim \mathcal{D}$

  $f_{\boldsymbol{\theta}^{(i)}}$ = `TrainClassifier`$(\mathcal{D}^{(i)})$

  $s^{(i)}$ = `GetRecourse`$(x, f_{\boldsymbol{\theta}^{(i)}})$

  teststats $\leftarrow T(s^i) = c(x, x'^{(i)})$

end for

train shadow models (only need to do once!) and recourses, collect their CFDs on input

ATTACK 2: CFD LRT

again, similar to preliminaries, but more adjustments

Algorithm 1 One-sided Distance-based Likelihood Ratio Test (`CFD LRT`)

Inputs: point (x, y), recourse output $s = \texttt{GetRecourse}(x, f_{\boldsymbol{\theta}}), \mathcal{D}$; FP-Rate: α, # Shadow Models: N, $\mathcal{T} = \texttt{TrainClassifier}(\cdot)$
teststats = []
Compute: $t_0 = T(s) = c(x, x')$ ← compute CFD on input
for $i = 1 : N$ **do**
    Sample $\mathcal{D}_t^{(i)} \sim \mathcal{D}$
    $f_{\boldsymbol{\theta}^{(i)}} = \texttt{TrainClassifier}(\mathcal{D}^{(i)})$
    $s^{(i)} = \texttt{GetRecourse}(x, f_{\boldsymbol{\theta}^{(i)}})$
    teststats $\leftarrow T(s^i) = c(x, x'^{(i)})$
end for
$\hat{\mu}_{\text{MLE}} = \frac{1}{N} \sum_{i=1}^{N} \left(\log c(x, \mathbf{x}'^{(i)}) \right)$
$\hat{\sigma}^2_{\text{MLE}} = \frac{1}{N} \sum_{i=1}^{N} \left(\hat{\mu}_{\text{MLE}} - \log \left(c(x, \mathbf{x}'^{(i)}) \right) \right)^2$

train shadow models (only need to do once!) and recourses, collect their CFDs on input

estimate params of normal distribution

ATTACK 2: CFD LRT

again, similar to preliminaries, but more adjustments

Algorithm 1 One-sided Distance-based Likelihood Ratio Test (`CFD LRT`)

Inputs: point (x, y), recourse output $s = \texttt{GetRecourse}(x, f_{\boldsymbol{\theta}}), \mathcal{D}$; FP-Rate: α, # Shadow Models: N, $\mathcal{T} = \texttt{TrainClassifier}(\cdot)$
teststats = [ ]
Compute: $t_0 = T(s) = c(x, x')$ ← compute CFD on input
for $i = 1 : N$ **do**
    Sample $\mathcal{D}_t^{(i)} \sim \mathcal{D}$
    $f_{\boldsymbol{\theta}^{(i)}} = \texttt{TrainClassifier}(\mathcal{D}^{(i)})$
    $s^{(i)} = \texttt{GetRecourse}(x, f_{\boldsymbol{\theta}^{(i)}})$
    teststats $\leftarrow T(s^i) = c(x, x'^{(i)})$
end for
$\hat{\mu}_{\text{MLE}} = \frac{1}{N} \sum_{i=1}^{N} \left(\log c(x, \mathbf{x}'^{(i)}) \right)$
$\hat{\sigma}^2_{\text{MLE}} = \frac{1}{N} \sum_{i=1}^{N} \left(\hat{\mu}_{\text{MLE}} - \log \left(c(x, \mathbf{x}'^{(i)}) \right) \right)^2$
if $t_0 > z_{1-\alpha}$ **then**
    Output: G = `NON-MEMBER`
else
    Output: G = `MEMBER`
end if

train shadow models (only need to do once!) and recourses, collect their CFDs on input

estimate params of normal distribution

▷ $z_{1-\alpha}$ is the $1-\alpha$-quantile of $Z \sim \mathcal{LN}(\hat{\mu}_{\text{MLE}}, \hat{\sigma}^2_{\text{MLE}})$

threshold τ

Is privacy leakage through recourses inevitable?

Is privacy leakage through recourses inevitable?

Privacy community thinks DP in training can bound the success of any adversary.

BOUNDING SUCCESS OF $\mathscr{A}$ WITH DP

Theorem 1. *Let $\mathcal{T} : (\mathcal{X} \times \mathcal{Y})^n \to \Theta$ denote the training algorithm, draw $D_t \sim \mathcal{D}^n$ and and $\mathcal{A}$ be an arbitrary adversary that receives $z = (x, y), s \sim \mathcal{R}(f_\theta, x, D_t)$ from the recourse inference game, and produces a guess $G \in \{MEMBER, NON\text{-}MEMBER\}$. Then, if $\mathcal{R}$ is $(\epsilon, 0)$-differentially private, we have for all $\mathcal{A}$:*

$$BA_{\mathcal{A}} \leq \frac{1}{2} + \frac{1 - e^{-\epsilon}}{2}.$$

Implications:

- Using DP in training, we can strongly bound the adversary's **balanced accuracy** success ((TPR + TNR) / 2) — not just excess accuracy broadly
- For a small FPR α, TPR of $\mathscr{A}$ is also close to α

BOUNDING SUCCESS OF $\mathscr{A}$ WITH DP

Theorem 1. *Let $\mathcal{T} : (\mathcal{X} \times \mathcal{Y})^n \to \Theta$ denote the training algorithm, draw $D_t \sim \mathcal{D}^n$ and and $\mathcal{A}$ be an arbitrary adversary that receives $z = (x, y), s \sim \mathcal{R}(f_\theta, x, D_t)$ from the recourse inference game, and produces a guess $G \in \{MEMBER, NON\text{-}MEMBER\}$. Then, if $\mathcal{R}$ is $(\epsilon, 0)$-differentially private, we have for all $\mathcal{A}$:*

$$BA_{\mathcal{A}} \leq \frac{1}{2} + \frac{1 - e^{-\epsilon}}{2}.$$

Implications:

- Using DP in training, we can strongly bound the adversary's **balanced accuracy** success ((TPR + TNR) / 2) — not just excess accuracy broadly
- For a small FPR α, TPR of $\mathscr{A}$ is also close to α

Proof:

- Appendix A; mostly applies definitions and expands integrals

Bounding success of $\mathscr{A}$ with DP

Theorem 1. *Let $\mathcal{T} : (\mathcal{X} \times \mathcal{Y})^n \to \Theta$ denote the training algorithm, draw $D_t \sim \mathcal{D}^n$ and and $\mathcal{A}$ be an arbitrary adversary that receives $z = (x, y), s \sim \mathcal{R}(f_\theta, x, D_t)$ from the recourse inference game, and produces a guess $G \in \{MEMBER, NON\text{-}MEMBER\}$. Then, if $\mathcal{R}$ is $(\epsilon, 0)$-differentially private, we have for all $\mathcal{A}$:*

$$BA_{\mathcal{A}} \leq \frac{1}{2} + \frac{1 - e^{-\epsilon}}{2}.$$

Implications:

- Using DP in training, we can strongly bound the adversary's **balanced accuracy** success ((TPR + TNR) / 2) — not just excess accuracy broadly
- For a small FPR α, TPR of $\mathscr{A}$ is also close to α

Proof:

- Appendix A; mostly applies definitions and expands integrals

However:

- DP is not a silver bullet! Training with DP causes significant drop in accuracy

Experimental Evaluation: Setup

Datasets

1. Adult (A)
   - Label: whether income > 50,000
2. Home Equity Line of Credit (H)
   - Label: whether individuals will repay HELOC
3. Diabetes (D)
   - Label: whether patient will be readmitted within next 30 days
4. Synthetic
   - Label: comes from Gaussian samples

Recourse Algorithms

1. SCFE (Wachter et al.)
   - Gradient-based objective
2. Growing Spheres (GS)
   - Random search in the input space
3. CCHVAE
   - Trains a variational autoencoder (VAE)
   - VAE searches in a lower-dimensional latent space

Experimental Evaluation: Procedure

Subsampling

- Subsampling 10,000 data points
- 5,000 points: owner trains private model
- 5,000 points: adversary trains shadow model, for CFD likelihood ratio (LRT) attack

Training

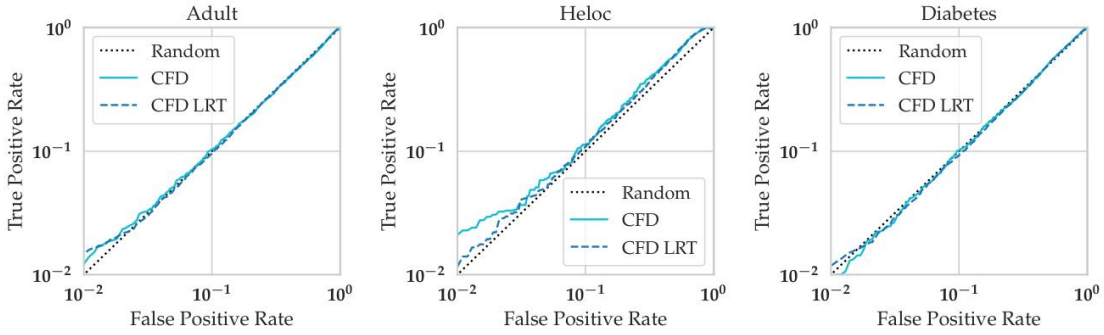
- Fully connected classifier neural network
- 1 hidden layer: 1000 nodes, ReLu activation
- ADAM optimizer (lr=0.0001)
- 250 epochs

Evaluation

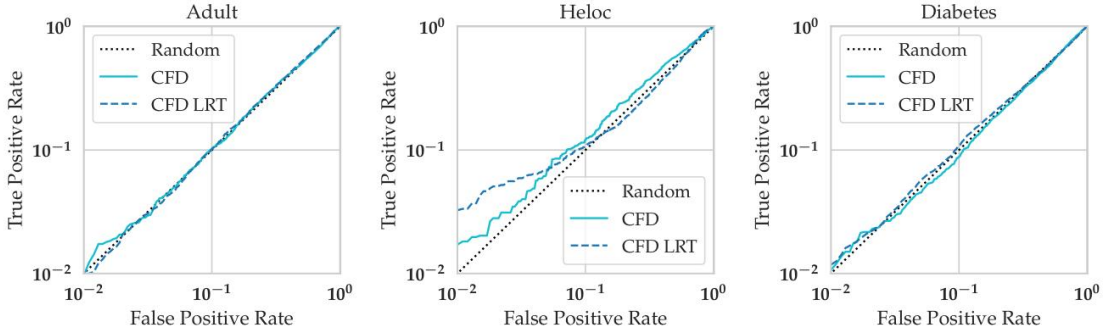
- Balanced accuracy
- Receiver operating characteristic AUC score
- Log-scale ROC curves
- True positive rates (TPRs) at low false positive rates (FPRs)

ATTACK EFFICIENCY

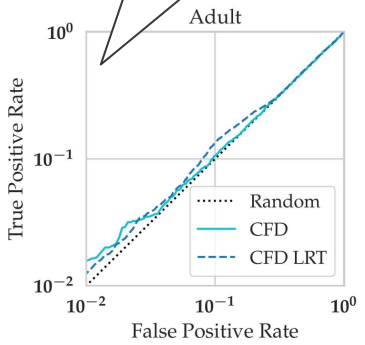
log-log transformation

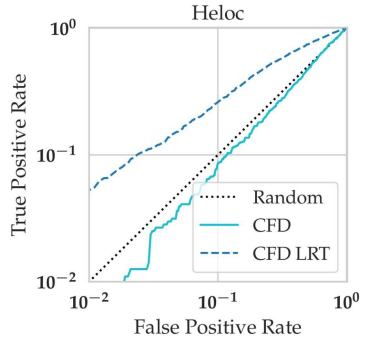


(a) SCFE



(b) GS





(c) CCHVAE

Attack Efficiency: Takeaways

Small upwards deviations from the diagonal at low FPRs (i.e. 0.01)

↓

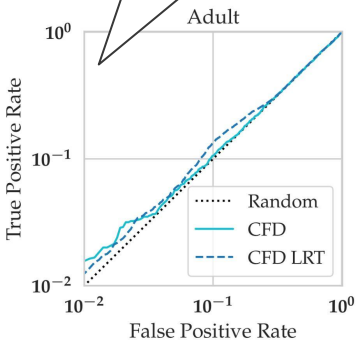
A small fraction of points are accurately identified as members

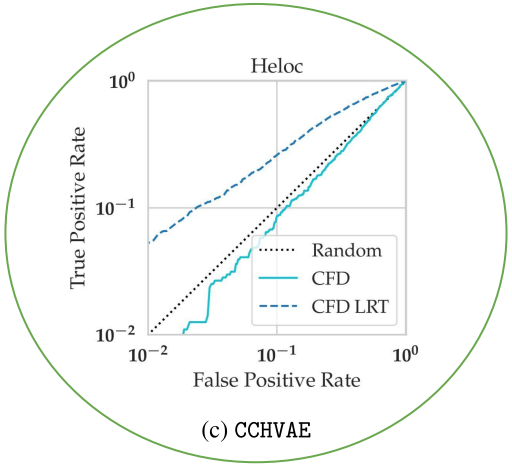
↓

Successful MI attack

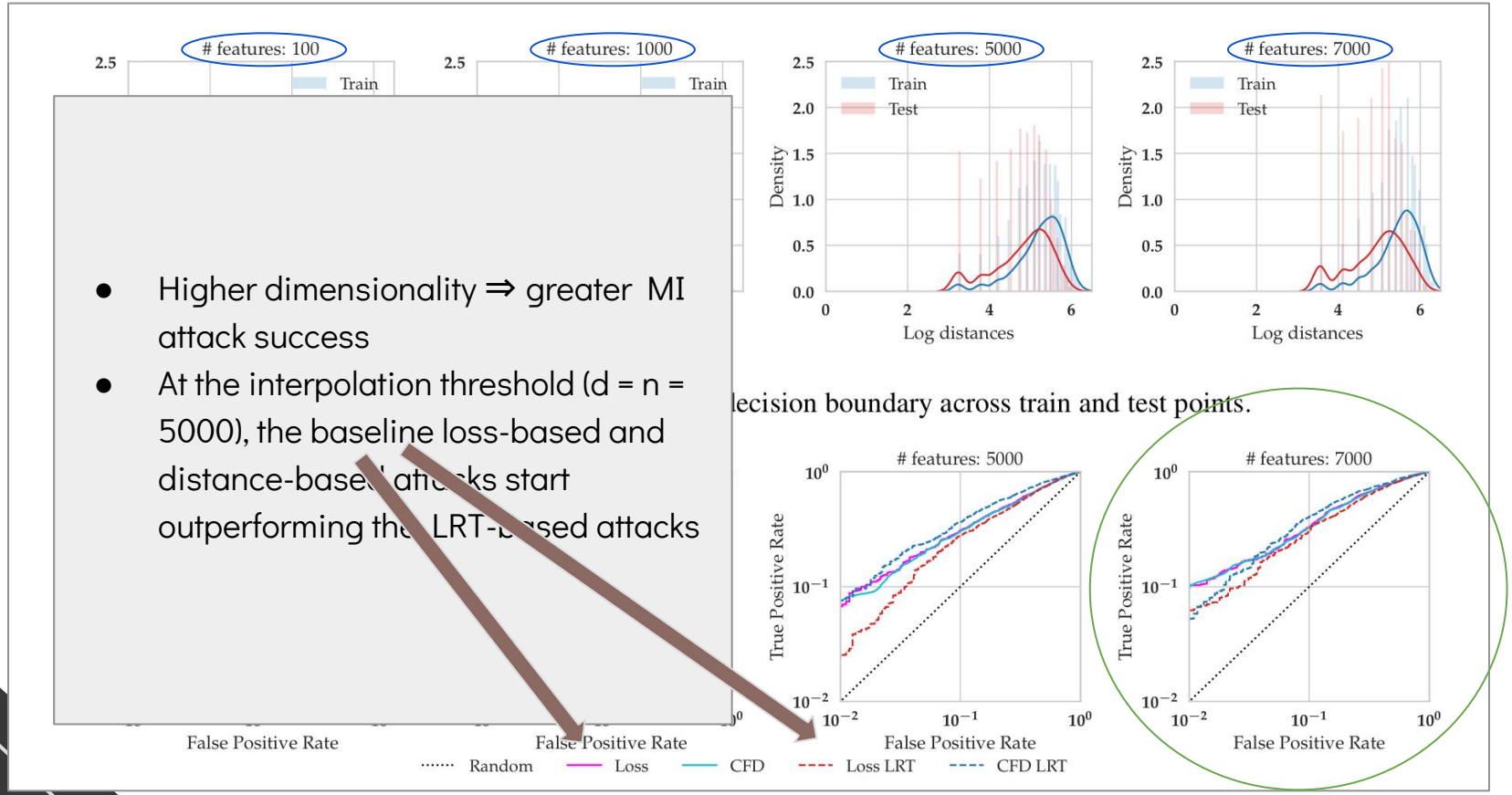
- Both methods (CFD, CFD LRT) often outperform the random baseline across all metrics
- CFD LRT generally outperforms CFD

log-log transformation





EFFECTS OF # FEATURES



- Higher dimensionality ⇒ greater MI attack success
- At the interpolation threshold (d = n = 5000), the baseline loss-based and distance-based attacks start outperforming the LRT-based attacks

Effects of Model Architecture

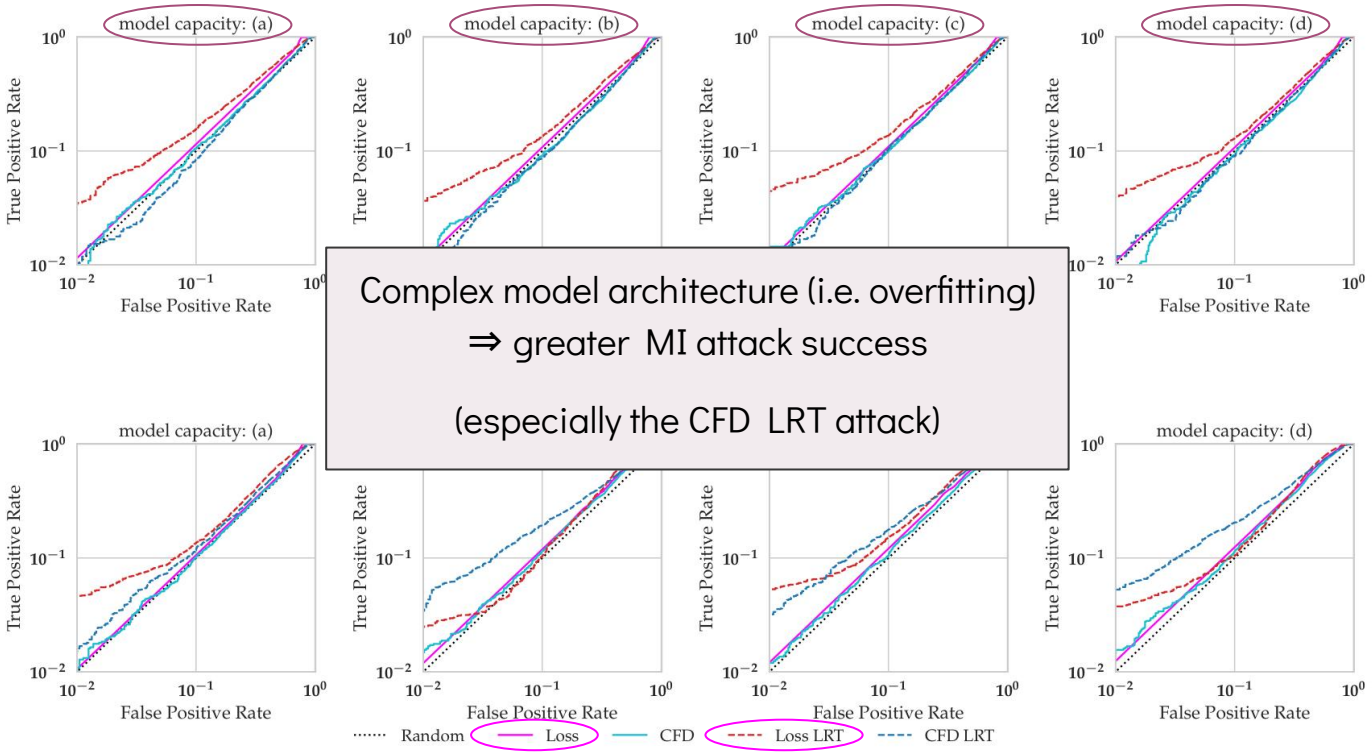
Models

(a) 2 layers, 1000 hidden notes

(b) 3 layers, 100 hidden nodes

(c) 3 layers, 333 hidden nodes

(d) 3 layers, 1000 hidden nodes



Complex model architecture (i.e. overfitting)
⇒ greater MI attack success

(especially the CFD LRT attack)

(b) # features = 150

When are CFD-based attacks most successful?

- When data dimensionality is high (# of features)
- When underlying model overfits to training data (model architecture)
- Combination of the above ⇒ increased vulnerability of recourses to CFD-based MI attacks

Conclusion

Novel Attacks

- **Idea:** we can leverage recourses to infer *private* training data membership information
- **Contribution:** MI attacks that leverage *counterfactual distances* output by recourse methods

Evidence of privacy Leakage

- **Implications:** privacy leakage is a risk of recourse algorithms; explainability-privacy tradeoff
- **Relevance:** proposed MI attacks are effective in diverse domains (lending, healthcare, law)

LIMITATIONS

- CFD is only a heuristic—an approximation of the distance of data point x to the model boundary
  - Recall: CFD = distance from x to its recourse
- Attacks operate under assumption that adversary can only query recourse algorithm once
- Must assume adversary knows optimal threshold that maximizes TPR given a fixed FPR
- This paper highlights a problem (privacy leakage), but not yet a solution
- Assessed on binary classification tasks only
  - Generalizability of results (broadly)

FUTURE WORK

- **Generalization:** whether recourse exposes us to other forms of privacy leakage
    - Can algorithmic recourse lead to successful reconstruction attacks? How about attacks on sensitive summary statistics of the training data (or anything else about the data distribution)?
- **Generalization:** which other XAI mechanisms involve privacy violations?
- **Solutions to protect privacy:** whether we can train models that provide recourse while mitigating privacy risks
    - How do we construct faithful model explanations that also do not leak too much information about the underlying training data? What is the privacy-utility trade-off of such models?

Discussion Questions

1. Given the explainability-privacy tradeoff highlighted in this paper, what is the role of each of the following in determining explainability and privacy benchmarks when training a model?
   a. ML practitioners (model builders and model breakers)
   b. End users (model consumers)
2. Both privacy and explainability can cultivate user *trust* in an ML model, and the lack thereof of any of these can break this trust. Both pillars are crucial but cannot fully coexist (this is the crux of our paper)—in what situations would you care about one pillar over the other?
3. Besides recourse, what other XAI mechanisms do you think might lead to privacy violations?
4. Is it even possible to have private explanations? Is this even worth going for?